

Stack & Queue Implementation using Structures (Self-Learning)

Q. Why is it advantageous to implement Stacks and Queues using Structures in C? Provide a C program demonstrating the implementation of a Stack and a Queue where their components are encapsulated within a structure.

📌 Definition to Remember

Implementing Stacks and Queues using a **Structure (struct)** bundles the data array and its control variables (e.g., **top** or **front** / **rear**) into a single, cohesive unit. This provides **encapsulation**, prevents global variable conflicts, and allows the creation of **multiple independent instances** of the data structure.

1. Advantages of Using Structures

- **Encapsulation:** Data array and control variables are logically grouped together.
- **Multiple Instances:** You can create **Stack s1, s2;** and manage them independently. Impossible if **top** and **array** are global variables.
- **Code Clarity (Pass-by-Reference):** Passing **Stack *s** to a function is cleaner and avoids copying large arrays in memory.

2. Stack Implementation using Structure

```
#include <stdio.h>
#define MAX 5

// Define the Stack Structure
typedef struct {
    int data[MAX];
    int top;
} Stack;

// Initialize stack
void initStack(Stack *s) {
    s->top = -1;
}

// Push operation
void push(Stack *s, int value) {
    if (s->top == MAX - 1) {
        printf("Stack Overflow\n");
    } else {
        s->data[++(s->top)] = value;
        printf("%d pushed.\n", value);
    }
}
```

3. Queue Implementation using Structure

```
#include <stdio.h>
#define MAX 5

// Define the Queue Structure
typedef struct {
    int data[MAX];
    int front;
    int rear;
} Queue;

// Initialize queue
void initQueue(Queue *q) {
    q->front = -1;
    q->rear = -1;
}

// Enqueue operation
void enqueue(Queue *q, int value) {
    if (q->rear == MAX - 1) {
        printf("Queue Overflow\n");
    } else {
        if (q->front == -1) q->front = 0;
        q->data[++(q->rear)] = value;
        printf("%d enqueued.\n", value);
    }
}
```

★ Must-Write Points (for 10 marks)

1. Standard arrays use global variables for **top** (Stack) or **front/rear** (Queue), limiting the program to one instance.
2. **struct** bundles the array and control variables into one cohesive unit (**Encapsulation**).
3. Allows creating **multiple independent instances** (e.g., **Stack s1, s2**).
4. We pass a **pointer to the structure** (**Stack *s**) to functions to modify the original instance directly.
5. Pointer usage (Pass-by-Reference) avoids memory-heavy copying of the array.
6. Stack Struct: contains **int data[MAX]** and **int top** .
7. Queue Struct: contains **int data[MAX]** , **int front** , and **int rear** .

⚡ Quick Recall

Implementation via Struct → Bundles Array + Pointers (top/front/rear) → Encapsulation → Allows multiple independent instances (s1, s2) → Pass-by-reference avoids array copying